

Coding Assignment 6

ECS 122A Algorithm Design and Analysis

Regular Expression Matching in ByteLand

Problem Description

In this assignment, you will implement a simplified regular expression matcher. You are given two strings T and P , called the *text string* and the *pattern string*, respectively. The text string contains only English letters. The pattern string may contain English letters and four types of special characters:

., *, +, ?.

Your task is to determine whether the entire text string matches the entire pattern string.

The special characters have the following meanings:

- A dot character “.” matches exactly one English letter.
- A star character “*” matches a (possibly empty) string of English letters.
- A plus character “+” matches a non-empty string of English letters.
- A question mark character “?” matches either the empty string or exactly one English letter.

English letters in the pattern are matched literally. That is, an English letter in the pattern must match the same English letter in the text. The matching is case-sensitive. For example, **A** and **a** are considered different characters.

Equivalently, your program should decide whether it is possible to transform the pattern string into the text string by applying the following operations an arbitrary number of times:

- Replace a dot character with exactly one English letter.
- Replace a star character with a (possibly empty) string of English letters.
- Replace a plus character with a non-empty string of English letters.
- Replace a question mark character with either the empty string or exactly one English letter.

Some examples are given below.

- If $T = P = \text{“DavisTheBest”}$, then the answer is YES.
- If $T = \text{“DavisTheBest”}$ and $P = \text{“Dav*The*”}$, then the answer is YES, because the first star can be replaced by “is” and the second star can be replaced by “Best”.
- If $T = \text{“DavisTheBest”}$ and $P = \text{“Davi.The*Bes.”}$, then the answer is YES. The dot after “Davi” matches the letter “s”, the star is replaced by the empty string, and the last dot matches the letter “t”.
- If $T = \text{“DavisTheBest”}$ and $P = \text{“DavisTheBest.”}$, then the answer is NO, because the last dot must match exactly one English letter.
- If $T = \text{“sampleABC”}$ and $P = \text{“sample+”}$, then the answer is YES, because the plus can be replaced by “ABC”.

- If $T = \text{"sample"}$ and $P = \text{"sample+"}$, then the answer is NO, because the plus must match a non-empty string.
- If $T = \text{"algorithm"}$ and $P = \text{"algo?ithm"}$, then the answer is YES, because the question mark can match the letter "r".
- If $T = \text{"algorithm"}$ and $P = \text{"algor?ithm"}$, then the answer is YES, because the question mark can match the empty string.

Submission Requirements

- You may use external sources that already exist on the web, but you must cite every such source you used and briefly state what you took from it. Use of LLM-based tools is **not allowed**.
- Some submissions may be selected for a brief interactive grading follow-up, in which you may be asked to explain the reasoning behind **your submitted** solution and the main ideas in your code. Since the assignment credit is meant to reflect understanding of the submitted work, a submission may receive reduced credit if, during such a follow-up, the solution cannot be adequately explained.

Input and Output

Each input file contains several test instances. The first line contains a single integer C , representing the number of test instances. The description of the C instances follows.

Each test instance consists of two lines:

- The first line contains the text string T .
- The second line contains the pattern string P .

For each test instance, your program should output a single line:

- Output YES if the text string matches the pattern string.
- Output NO otherwise.

In this assignment, you are provided with sample submissions that contain code for input and output. You only need to complete a function that, given the text string and the pattern string, returns whether the text matches the pattern.

Constraints

Each input file contains at most 2000 test instances. In a single input file, the total length of all text strings across all test instances is at most 2000, and the total length of all pattern strings across all test instances is at most 2000. The time limit is 1 second for C/C++ and 3 seconds for Python.

Each test instance satisfies the following additional constraints:

- $1 \leq |T| \leq 2000$.
- $1 \leq |P| \leq 2000$.
- The text string T contains only English letters.
- The pattern string P contains only English letters and the special characters `.`, `*`, `+`, and `?`.

Remark 1: The special characters in this assignment are not used in the same way as in standard regular expressions in many programming languages. In particular, the characters `*`, `+`, and `?` are standalone wildcard characters; they do not modify the preceding character.

Test Cases

Your program will be evaluated on 10 test files. Passing all the instances within a file will earn you 0.8 points. Note that if your program outputs an incorrect answer for even one of the instances within a file, you will receive 0 points for that file.

Subtasks

Out of the 10 test files:

- 6 test files will contain only dot characters as special characters. That is, the pattern strings in these files may contain English letters and ., but they will not contain *, +, or ?.
- 2 additional test files will contain only dot and question mark characters as special characters. That is, the pattern strings in these files may contain English letters, ., and ?, but they will not contain * or +.
- The remaining 2 test files may contain all four special characters: ., *, +, and ?.

Sample Input 1:

```
8
DavisTheBest
DavisTheBest
DavisTheBest
Dav*The*
DavisTheBest
Davi.The*Bes.
DavisTheBest
DavisTheBest.
sampleABC
sample+
sample
sample+
algorithm
algo?ithm
algorithm
algor?ithm
```

Sample Output 1:

```
YES
YES
YES
NO
YES
NO
YES
YES
```

Sample Input 2:

```
6
ABC
A.C
ABC
A.?C
ABBC
A*C
AC
```

```
A+C
AXYZB
A+B
AB
A+B
```

Sample Output 2:

```
YES
YES
YES
NO
YES
NO
```

Sample Explanation

In Sample Input 1:

- The first instance matches because the text and the pattern are identical.
- The second instance matches because the two stars can be replaced by “is” and “Best”, respectively.
- The third instance matches because the first dot matches “s”, the star matches the empty string, and the last dot matches “t”.
- The fourth instance does not match because the final dot requires one additional English letter.
- The fifth instance matches because the plus can be replaced by “ABC”.
- The sixth instance does not match because the plus must be replaced by a non-empty string.
- The seventh instance matches because the question mark can match the letter “r”.
- The eighth instance matches because the question mark can match the empty string.

In Sample Input 2:

- The pattern “A.C” matches “ABC” because the dot matches “B”.
- The pattern “A.?C” matches “ABC” because the dot matches “B” and the question mark matches the empty string.
- The pattern “A*C” matches “ABBC” because the star can be replaced by “BB”.
- The pattern “A+C” does not match “AC” because the plus must match a non-empty string.
- The pattern “A+B” matches “AXYZB” because the plus can be replaced by “XYZ”.
- The pattern “A+B” does not match “AB” because there is no letter between “A” and “B” for the plus to match.

Submission Guideline

Write your program in either C, C++, or Python **in a single file**. Submit the file on Gradescope. The time limit on Gradescope is 1 second for C/C++ and 3 seconds for Python. You can make at most 10 submission attempts. You may refer to `sample-cpp.cpp` or `sample-py.py` for sample code that takes input and writes output.